LESSON PLAN · PYTHON (GR 6–8) BEGINNER: PYTHON GAME CODERS

Lesson 8: Capstone: Treasure Hunt

Module 2 | 60 minutes | game design, lists, def | Grades 6–10 · ages 11–15

Objective: Design and build a text adventure game with rooms, choices, an inventory list, win and lose endings, and your first functions.

Success criteria

- I can plan a game with a map, a win rule and a lose rule.
- I can use a while loop, a room variable and if / elif to run a text adventure.
- I can use an inventory list with append and in.
- I can define and call a simple function with def.

Key vocabulary

- **text adventure**: A game where the player types commands to explore a story.
- **inventory**: A list of items the player is carrying.
- **function**: A named block of code you can run again and again.
- **def**: The word that defines a new function.
- **call**: Running a function by writing its name with brackets.
- **playtest**: Letting someone play your game to find problems.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Printed project brief
- Project files: room_describer.py, treasure_hunt_map.md

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Dungeon Master**	Play Dungeon Master (unplugged) in the chat. Ask what the “program” had to remember.
5 min	**Hook, video and map**	Show the hook and play the capstone video with captions. Thumbs poll: “Who has played a text adventure?” Learners draw their map on paper.
8 min	**Live demo and build-along**	Run the live demo, then everyone builds the room describer functions. Predict-the-output and the design-order activity as a quick competition.
20 min	**Build the Treasure Hunt**	Learners build their adventure from the starter, one room at a time. Halfway check-in: the loop, quit and one room work. Teacher visits breakout rooms.

5 min	**Debug race**	Race to fix “the broken adventure”: three bugs. Winner explains why the call had to move.
9 min	**Playtest swap and showcase**	Breakout pairs play each other’s adventure and give two stars and a wish. Back together, three learners run a live showcase while the class calls out commands.
5 min	**Improve and submit**	Fix one thing from playtesting, write the reflection and submit.
3 min	**Celebrate and exit ticket**	Course celebration: everyone shares one emoji for their favourite project. Exit ticket: append and check "map".

Unplugged warm-up

Dungeon Master (5 min game): the teacher describes a room (“You are in a dark hall. Doors lead north and east.”) and learners type commands in the chat. The teacher only accepts exact commands and keeps an inventory list on the board. After 3 rooms, ask: what did the “program” need to remember? (the room, the inventory, the moves).

Teaching the idea

The capstone pulls the whole course together: variables for the room and moves, input for commands, if/elif for choices, a while loop for the game, a list for the inventory, and a first taste of functions. A function is a named block of code defined with def and run by calling its name with brackets. Insist on design first: a paper map with rooms, the key, the treasure, the win rule and the lose rule. Then build the loop and ONE room, test, and add rooms one by one.

Analogy: A function is like a recipe card in the kitchen: you write “make chai” once, and whenever you need chai you just say “make chai” instead of reading every step again.

Live demo: 1) Type def line(): with an indented print("=" * 30) and run: nothing appears. Ask why. 2) Add line() at the left edge and run. 3) Type def describe_room(name): with print(f"You are in the {name}."), then call it with "Hall" and "Cave". 4) Type inventory = [], inventory.append("key"), and if "key" in inventory: print("The door opens!"). 5) Open the Treasure Hunt starter and add two rooms live: elif room == "hall" and cmd == "north": room = "library". Play it with the class calling out commands. 6) Add the torch rule (if playing and moves >= 10: print("GAME OVER")) and show the lose ending by typing nonsense 10 times. Winning route for the model solution: north, take, south, east, down, take.

Questions to ask

Ask	Listen for
What does def do?	It defines (creates) a new function.
Why did nothing happen after we defined line()?	A function only runs when you call it.
How does the game remember where the player is?	With a variable called room.
How do we check if the player has the key?	if "key" in inventory:

How does the game end?	Set playing = False, so the while loop stops.
Why test one room at a time?	Bugs stay small and easy to find.

Differentiation

Support
Give the full starter with the loop and quit already working; learners add rooms using the one-room pattern card. Allow 2 rooms plus the treasure room as a first goal.

Stretch
Add a second item (a torch refill that adds 5 moves), a trap room that ends the game, or a describe(place) function that prints every room description.

Common misconceptions

Misconception	How to fix it
Defining a function runs it	You must call it by name with brackets, like show_status().
You can call a function above its def line	Python reads top to bottom, so define it first.
elif cmd == "north" works in every room	Check the room too: room == "hall" and cmd == "north".
while playing stops by itself	Something must set playing = False: winning, losing or quit.

Assessment

Project rubric (also on the project brief):

Criterion	1 · Beginning	2 · Developing	3 · Secure	4 · Excellent
Game design	No clear map or goal	A goal but rooms do not connect clearly	3+ connected rooms, a locked door, win and lose rules	A clever, well-described world with a puzzle that makes the player think
Programming	Crashes or the loop never ends	Loop works but some rooms or items do not	Rooms, inventory, win, lose and quit all work	Also uses functions well, e.g. describe(place), and handles odd input
Lists and functions	No inventory list or function	A list or a function, but not used properly	Inventory with append and in; at least one function called	Several functions that make the code shorter and clearer
Testing and presentation	Not tested	Tested by the maker only	Playtested by someone else and one fix made	Clear instructions, comments, a playtest log and a confident showcase

Exit ticket
Write the two lines that add "map" to the inventory list and then check whether "map" is in it.

Homework
Get two people at home to play your Treasure Hunt without help. Write down where they got stuck, then improve that room’s description.

Answer key — Kitabash activities

1. Python Lab: room describer (perform · python lab)

Goals checked: Describes all three rooms · Calls describe_room 3 times · Defines a function

One solution:

```
# Room describer: my first functions
def line():
    print("=" * 30)

def describe_room(name):
    line()
    print(f"You are in the {name}.")
    line()

describe_room("Hall")
describe_room("Kitchen")
describe_room("Cave")
```

2. Predict the output: what is in the bag? (perform · code output)

3. Design a game (perform · ordering)

Order: Draw a map of the rooms → Choose where the key and treasure go → Write the win and lose rules → Code the game loop and one room → Add the other rooms and test each one → Playtest with a friend and improve

4. Capstone: Treasure Hunt (perform · python lab)

Goals checked: Typing quit ends the game · Torch runs out after 10 moves · Has rooms, an inventory and a function

Example program:

```
# Treasure Hunt: a text adventure
# Made by: Zoya
# Map: hall -north-> library, hall -east-> kitchen, kitchen -down-> cellar (needs the
key)

inventory = []
room = "hall"
moves = 0
max_moves = 10
playing = True

def show_status():
    print("-" * 34)
    print(f"Room: {room} | Moves left: {max_moves - moves}")
    print("Bag:", inventory)

def describe(place):
    if place == "hall":
        print("A dusty hall. Doors lead north and east.")
    elif place == "library":
        print("Old books everywhere. Something shiny is on the desk. Exit: south.")
    elif place == "kitchen":
        print("A smoky kitchen. A trapdoor leads down. Exit: west.")
    elif place == "cellar":
        print("A dark cellar. A wooden chest sits in the corner. Exit: up.")

print("=== TREASURE HUNT ===")
print("Find the treasure before your torch runs out!")
print("Commands: north, south, east, west, up, down, take, quit")

while playing:
    show_status()
    describe(room)
    cmd = input("> ").strip().lower()
    moves += 1

    if cmd == "quit":
        print("Bye! Thanks for playing.")
        playing = False
    elif room == "hall" and cmd == "north":
        room = "library"
    elif room == "hall" and cmd == "east":
        room = "kitchen"
    elif room == "library" and cmd == "south":
        room = "hall"
    elif room == "library" and cmd == "take":
        if "key" in inventory:
            print("You already have the key.")
        else:
            inventory.append("key")
            print("You picked up a golden key!")
```

```
    elif room == "kitchen" and cmd == "west":
        room = "hall"
    elif room == "kitchen" and cmd == "down":
        if "key" in inventory:
            print("The key opens the trapdoor...")
            room = "cellar"
        else:
            print("The trapdoor is locked. You need a key.")
    elif room == "cellar" and cmd == "up":
        room = "kitchen"
    elif room == "cellar" and cmd == "take":
        inventory.append("treasure")
        print("You open the chest and find the TREASURE! YOU WIN!")
        playing = False
    else:
        print("You can't do that here.")

    if playing and moves >= max_moves:
        print("Your torch flickers out. GAME OVER!")
        playing = False
```

5. Debug it: the broken adventure (probe · python lab)

Goals checked: Runs with no errors · Prints both lines in order · Still uses the function

One solution:

```
def greet(name):
    print(f"Welcome, {name}!")

greet("Explorer")

bag = []
bag.append("torch")
if "torch" in bag:
    print("You can see in the dark.")
```

Answer key — worksheet

1. **Key words:** text adventure = A game where the player types commands to explore a story. · inventory = A list of items the player is carrying. · function = A named block of code you can run again and again. · def = The word that defines a new function.
2. **Plan your program:** Goals: Describes all three rooms; Calls describe_room 3 times; Defines a function. One solution: # Room describer: my first functions def line(): print("=" * 30) def describe_room(name): line() print(f"You are in the {name}.") line() describe_room("Hall") describe_room("Kitchen") describe_room("Cave")
3. **Predict the output: what is in the bag?:** A. You have 2 things.
4. **Design a game:** 1 Draw a map of the rooms · 2 Choose where the key and treasure go · 3 Write the win and lose rules · 4 Code the game loop and one room · 5 Add the other rooms and test each one · 6 Playtest with a friend and improve

5. **Plan your program:** Goals: Runs with no errors; Prints both lines in order; Still uses the function. One solution: def greet(name): print(f"Welcome, {name}!") greet("Explorer") bag = [] bag.append("torch") if "torch" in bag: print("You can see in the dark.")
6. **My plan:** A clear plan that covers every requirement.
7. **Checklist:** All requirements met.
8. **Two stars and a wish:** Kind, specific feedback and an improvement.

Kitabash original curriculum.